

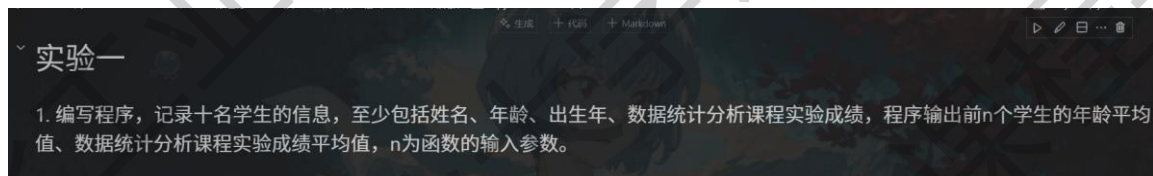
实验项目名称	实验一 基本语法及运算		实验成绩	
实 验 者	艾洋	专业班级	数据科学与大数据技术 数据 231	
实验日期			2025 年 11 月 21 日	

一、实验内容

- 1、编写程序，记录十名学生的信息，至少包括姓名、年龄、出生年、数据统计分析课程实验成绩，程序输出前 n 个学生的年龄平均值、数据统计分析课程实验成绩平均值，n 为函数的输入参数。
- 2、找出 1 至 999 之间是 13 的倍数或者前两位数字是 13 的数字，输出这些数字，并统计有多少个。
- 3、编写成绩转化为绩点的函数，用 98, 93, 89, 73, 66 分别调用函数，生成对应绩点。
- 4、随机生成两个长为 100 且服从标准正态分布的向量，然后将两向量所有偶数位的数值对调，输出所有的 4 个向量（对调前的 2 个，对调后的 2 个）。
- 5、已知 $XYZ + YZZ = 532$ ，其中 X、Y 和 Z 为数字，编程求出 X，Y 和 Z 的值。

二、实验源程序及结果截图

1. 源程序核心代码：



```
students_list = [
    {"name": "张三", "age": 20, "birth_year": 2004, "grade": 88},
    {"name": "李四", "age": 21, "birth_year": 2003, "grade": 92},
    {"name": "王五", "age": 19, "birth_year": 2005, "grade": 76},
    {"name": "赵六", "age": 20, "birth_year": 2004, "grade": 95},
    {"name": "孙七", "age": 22, "birth_year": 2002, "grade": 81},
    {"name": "周八", "age": 21, "birth_year": 2003, "grade": 89},
    {"name": "吴九", "age": 19, "birth_year": 2005, "grade": 90},
    {"name": "郑十", "age": 20, "birth_year": 2004, "grade": 78},
```

```

{"name": "陈十一", "age": 21, "birth_year": 2003, "grade": 85},
{"name": "刘十二", "age": 22, "birth_year": 2002, "grade": 93}
]

def calculation_avg(list,n):
    if n<=0:
        print("n 必须是正整数")
        return (0, 0)

    if n>len(list):
        print(f"n 值大于列表长度, 将会计算 {n} 个学生的平均")

    total_age = 0
    total_grade = 0
    for student in list[:n]:# student 是一个字典
        total_age+=student["age"]
        total_grade+=student["grade"]

    return total_age/n, total_grade/n# 返回元组, 可以加()

n_value=5

avg_age, avg_grade=calculation_avg(students_list,n_value)

print(f"前 {n_value} 个学生的平均年龄: {avg_age:.2f}")
print(f"前 {n_value} 个学生的平均成绩: {avg_grade:.2f}")

```

2. 找出1至999之间是13的倍数或者前两位数字是13的数字, 输出这些数字, 并统计有多少个。

```

def find():
    count=0

```

```

for i in range(1,1000):
    if i>=130 and i<=139 or i%13==0:# and 优先级高于 or
        print(f"{i}符合条件")
        count+=1
print(f"符合条件的数字有{count}个")

find()

```

3. 编写成绩转化为绩点的函数，用98, 93, 89, 73, 66分别调用函数，生成对应绩点。

```

def convert(score):
    if score>=90:
        return 4.0
    if score<60:
        print("成绩不及格")
        return 0
    return 4-(90-score)*0.1

score_list=[98,93,89,73,66]

for i in score_list:
    gpa=convert(i)
    print(f"{i}的绩点为{gpa:.1f}")# 不写.f 会默认保留6位小数（计算机无法精确存储0.1）

```

4. 随机生成两个长为100且服从标准正态分布的向量，然后将两向量所有偶数位的数值对调，输出所有的4个向量（对调前的2个，对调后的2个）。

```

import numpy as np
vector1=np.random.randn(100)# randn 生成服从标准正态分布的随机数，一般用 rand 生成[0,1)之间的随机数
vector2=np.random.randn(100)# 形状为(100,)，改成 randn(1,100), 形状就是

```

```

(1,100)
print(f"vector1:{vector1[:10]}")# 因为显示不全，所以只取前 10 个元素展示一下
print(f"vector2:{vector2[:10]}")

# : [start : stop] 默认 1 选取一个连续的子序列。
# :: [start : stop : step]
temp=vector1[0::2]
vector1[0::2]=vector2[0::2]
vector2[0::2]=temp

print(f"vector1:{vector1[:10]}")
print(f"vector2:{vector2[:10]}")

```

5. 已知 $XYZ+YZZ=532$ ，其中X、Y和Z为数字，编程求出X、Y和Z的值。

```

def solve():
    for x in range(10):
        for y in range(10):
            for z in range(10):
                if x*100+y*10+z+y*100+z*10+z==532:
                    print(f"x={x},y={y},z={z}")
                    return x,y,z

solve()

```

2. 运行结果（截图需同时截入右下角电脑时间）:

```
def calculation_avg(list,n):
    if n<=0:
        print("n 必须是正整数")
        return (0,0)

    if n>len(list):
        print(f"n值大于列表长度,将会计算{n}个学生的平均")

    total_age = 0
    total_grade = 0
    for student in list[:n]:# student是一个字典
        total_age+=student["age"]
        total_grade+=student["grade"]

    return total_age/n,total_grade/n# 返回元组,可以加()

n_value=5

avg_age,avg_grade=calculation_avg(students_list,n_value)

print(f"前{n_value}个学生的平均年龄: {avg_age:.2f}")
print(f"前{n_value}个学生的平均成绩: {avg_grade:.2f}")

[21] ✓ 0.0s
...
前5个学生的平均年龄: 20.40
前5个学生的平均成绩: 86.40

2. 找出1至999之间是13的倍数或者前两位数字是13的数字,输出这些数字,并统计有多少个。

def find():
    count=0
    for i in range(1,1000):
        if i>=130 and i<=139 or i%13==0:# and优先级高于or
            # print(f"{i}符合条件")
            count+=1
    print(f"符合条件的数字有{count}个")

find()

[26] ✓ 0.0s
...
符合条件的数字有85个

3. 编写成绩转化为绩点的函数,用98, 93, 89, 73, 66分别调用函数,生成对应绩点。

def convert(score):
    if score>=90:
        return 4.0
    if score<60:
        print("成绩不及格")
        return 0
    return 4-(90-score)*0.1

score_list=[98,93,89,73,66]

for i in score_list:
    gpa=convert(i)
    print(f"{i}的绩点为{gpa:.1f}")# 不写.f会默认保留6位小数(计算机无法精确存储0.1)

[27] ✓ 0.0s
...
98的绩点为4.0
93的绩点为4.0
89的绩点为3.9
73的绩点为2.3
66的绩点为1.6
```

The first screenshot shows a Python script in a Jupyter Notebook environment. The code imports numpy as np and generates two random vectors, vector1 and vector2, each of size 100. It then prints the first 10 elements of both vectors. Below this, it demonstrates slicing and assignment: it selects a subsequence from vector1 (indices 0 to 2) and assigns it to vector2 (indices 0 to 2), then prints the updated vectors.

```
import numpy as np
vector1=np.random.randn(100)# randn生成服从标准正态分布的随机数，一般用rand生成[0,1)之间的随机数
vector2=np.random.randn(100)# 形状为(100,)，改成randn(1,100),形状就是 (1,100)
print(f"vector1:{vector1[:10]}")# 因为显示不全，所以只取前10个元素展示一下
print(f"vector2:{vector2[:10]}")

# : [start : stop] 默认 1 选取一个连续的子序列。
# :: [start : stop : step]
temp=vector1[0::2]
vector1[0::2]=vector2[0::2]
vector2[0::2]=temp

print(f"vector1:{vector1[:10]}")
print(f"vector2:{vector2[:10]}")
```

The second screenshot shows a Python script in a Jupyter Notebook environment. It defines a function solve() that iterates over all possible values of x, y, and z (from 0 to 9) to find a solution to the equation x*100 + y*10 + z = 532. The function prints the solution and returns the values of x, y, and z. The output shows the solution (3, 2, 1).

```
def solve():
    for x in range(10):
        for y in range(10):
            for z in range(10):
                if x*100+y*10+z==532:
                    print(f"x={x},y={y},z={z}")
                    return x,y,z

solve()

x=3,y=2,z=1

(3, 2, 1)
```

三、实验的分析与思考（若有）

第一部分：数据结构与函数逻辑的构建

在处理学生信息管理的任务中，选择了使用**列表嵌套字典**的方式来存储数据。这个结构很棒，列表提供了元素的有序排列，而字典则允许通过清晰的键（如“age”、“grade”）来访问和管理每个学生的属性，极大地提高了代码的可读性和可维护性。在函数调用环节，我确认了Python 中 **元组解包（Tuple Unpacking）** 的高效性，即使用 `a, b = func()` 即可快速接收多重返回值。

对于函数内部的条件判断，尤其是在处理 `n` 值边界时，采用了更符合 Python 惯例的“**卫语句**”（Guard Clauses）结构，即使用独立的 `if` 语句配合 `return` 关键字实现“**Early Exit**”。我发现这种风格优于传统语言中严格的 `if...elif...else` 结构，能够让函数的后续主体逻辑更加清晰、扁平化，只专注于处理有效的数据范围。

第二部分：Python 语言特性与运算符的理解

在探讨逻辑运算符时，我明确了 Python 的设计哲学：它使用 `and`、`or`、`not` 等英文单词作为逻辑运算符，而非像 C 语言那样的符号（`&&`、`||`），我觉得这样确实增强了代码的自然

语言可读性。同时，我还复习了 Python 核心数据结构的区别：**列表** `[]` 是有序的序列，通过索引访问；而 **字典** `{}` 是通过键（Key）访问的映射关系。理解二者的差异是我高效进行数据存储和检索的前提。此外，在处理复杂的布尔表达式时，我确认了 `and` 运算符的优先级高于 `or`。

第三部分：数值计算与精度处理

在绩点转换的实验中，发现了计算机科学中处理**浮点数精度**的固有问题，即像 0.1 这样的数在二进制存储时会产生微小误差，导致 1.6 显示为 1.5999...。我发现解决此问题最简单的方式是在输出时使用 **f-string 格式化**来控制小数的显示位数。

第四部分：向量化操作与数组切片

在随机生成向量和执行对调操作的实验中，我区分了 NumPy 中的 `np.random.randn()`（标准正态分布）和 `np.random.rand()`（均匀分布）的使用场景，并掌握了 `np.random.rand(1, 100)` 生成**二维行向量**与 `np.random.rand(100)` 生成**一维向量**在形状上的重要区别。

最重要的是，我利用了 NumPy **切片**的功能。通过 `[0::2]` 语法（从索引 0 开始，步长为 2），我可以一次性选取和赋值所有偶数索引位置上的元素。这种批量操作的能力是 NumPy 高效性的体现。通过使用**临时变量** `temp` 完成任务。

实验项目名称	实验二 可视化绘图		实验成绩	
实 验 者	艾洋	专业班级	数据科学与大数据技术 数据 231	
实验日期			2025 年 11 月 21 日	

一、实验内容

- 1、一幅图内同时绘制 $\sin$ 、 $\cos$ 、 $\tan$ 函数曲线图，区间长度为 I ，三条曲线线型、颜色各不相同。
- 2、绘制三维图形 $x = \sin t - t \cos t$, $y = \cos t - t \sin t$, $z = t$, t 区间为 I ，图形 x 、 y 、 z 轴均加网格线， z 轴取对数分布。
- 3、随机生成一组有效的 50 人班级的成绩，并绘制合理的统计直方图。
提示：1、成绩为整数；2、直方图需选择合适区间。
- 4、统计自己十月份的各项支出，并利用软件绘制自己的支出饼状图，采用 3 种以上方式展示。
- 5、绘制散点图 $x = \sin t - t \cos t$, $y = \cos t - t \sin t$, t 区间为 I 。
- 6、绘制曲线图、阶梯图 $x = \sin t - t \cos t$, $y = \cos t - t \sin t$, t 区间为 I 。

二、实验源程序及结果截图

1. 源程序核心代码：

```

实验二

1.一幅图内同时绘制sin、cos、tan函数曲线图，区间长度为I，三条曲线线型、颜色各不相同。

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.ticker import ScalarFormatter
plt.rcParams['font.sans-serif'] = ['SimHei']

# 定义区间
x_start=235127-15
x_end=235127+15
# 生成 x
x=np.linspace(x_start,x_end,500)

```

```

# 定义 y
y_sin=np.sin(x)
y_cos=np.cos(x)
y_tan=np.tan(x)

plt.figure(figsize=(12, 6))
plt.plot(x,y_sin,label="sin(x)",color="red",linestyle="--")
plt.plot(x,y_cos,label="cos(x)",color="blue",linestyle="--")
plt.plot(x,y_tan,label="tan(x)",color="green",linestyle=":")

plt.title("实验 2-1: sin, cos, tan 函数曲线图", fontsize=16)
plt.xlabel("x", fontsize=14)
plt.ylabel("y", fontsize=14)

plt.legend()
plt.grid(True, alpha=0.6)# 网格线透明度

#这段为了避免科学计数法显示
ax = plt.gca()
formatter = ScalarFormatter(useOffset=False)
formatter.set_scientific(False)
ax.xaxis.set_major_formatter(formatter)

plt.ylim(-5, 5)
plt.show()

```

2. 绘制三维图形 $x = \sin t - t \cos t$, $y = \cos t - t \sin t$, $z = t$, t 区间为 l , 图形 x 、 y 、 z 轴均加网格线, z 轴取对数分布。

```

import numpy as np
import matplotlib.pyplot as plt

```

```

from mpl_toolkits.mplot3d import Axes3D
from matplotlib.ticker import ScalarFormatter # 导入格式化工具, 为了让 z
显示正确

t_start = 235112
t_end = 235142
t = np.linspace(t_start, t_end, 500)

x = np.sin(t) - t * np.cos(t)
y = np.cos(t) - t * np.sin(t)
z = t

fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

ax.plot(x, y, z, color='blue')

ax.set_xlabel('X 轴:  $\sin(t) - t\cos(t)$ ')
ax.set_ylabel('Y 轴:  $\cos(t) - t\sin(t)$ ')
ax.set_zlabel('Z 轴:  $t$ ')
ax.set_title(f'实验 2-2: 螺旋线', fontsize=16)

formatter = ScalarFormatter(useOffset=False) # 禁用偏移量
formatter.set_scientific(False) # 确保也不使用科学计数法

ax.zaxis.set_major_formatter(formatter)
plt.show()

```

3、随机生成一组有效的50人班级的成绩，并绘制合理的统计直方图。提示：1、成绩为整数；2、直方图需选择合适区间。

```

import numpy as np

```

```

import matplotlib.pyplot as plt

num_students = 50
mean_score = 75 # 目标平均分为 75
std_dev = 10    # 目标标准差为 10
# 生成 50 个标准正态分布 (均值为 0, 标准差为 1)
standard_normal_scores = np.random.randn(num_students)

# 将其平移和缩放, 得到分布 (均值为 75, 标准差为 10)
grades = standard_normal_scores * std_dev + mean_score

# 确保分数在 0 到 100 之间 (np.clip)
grades = np.clip(grades, 0, 100)
# 确保成绩为整数 (astype)
grades = grades.astype(int)

print(f"生成的 50 个整数成绩: \n{grades}")

plt.figure(figsize=(10, 6))

# 选择合适区间 (bins)
bins = [0, 60, 70, 80, 90, 101] # 0-59, 60-69, 70-79, 80-89, 90-100

# 绘制直方图 (并获取返回值以便后续添加标签)
counts, edges, _ = plt.hist(grades,
                             bins=bins,
                             color='steelblue',
                             edgecolor='black',
                             alpha=0.75)

```

```

plt.title('实验 2-3: 50 人班级成绩分布直方图', fontsize=16)
plt.xlabel('分数区间', fontsize=12)
plt.ylabel('学生人数 (频数)', fontsize=12)

# 设置 X 轴的刻度标签
plt.xticks(bins)

# 添加网格线
plt.grid(axis='y', linestyle='--', alpha=0.7)

# 在每个条形图上显示具体人数
bin_centers = 0.5 * (edges[:-1] + edges[1:]) # 计算每个区间的中心点
for count, x in zip(counts, bin_centers):
    if count > 0:
        plt.text(x, count + 0.2, str(int(count)), ha='center',
va='bottom')

plt.show()

```

4. 统计自己十月份的各项支出，并利用软件绘制自己的支出饼状图，采用3种以上方式展示。

```

import matplotlib.pyplot as plt

labels = ['餐饮美食', '购物消费', '交通出行', '休闲娱乐', '其他支出']
sizes = [1200, 300, 200, 200, 100]
colors = ['#ff9999', '#99ff99', '#ffcc99', '#c2c2f0', '#ffb3e6']

# 函数式编程风格
plt.figure(figsize=(20, 7))

```

```

plt.subplot(1, 3, 1)
plt.pie(sizes, labels=labels, colors=colors)
plt.title('方式一：标准饼状图')

plt.subplot(1, 3, 2)
explode = (0.1, 0, 0, 0, 0)
plt.pie(sizes, labels=labels, colors=colors, explode=explode)
plt.title('方式二：百分比图')

plt.subplot(1, 3, 3)
plt.pie(sizes, labels=labels, colors=colors, wedgeprops=dict(width=0.4))
plt.title('方式三：环形图')

plt.suptitle('实验 2-4：十月份个人支出分析', fontsize=20)
plt.show()

import matplotlib.pyplot as plt

labels = ['餐饮美食', '购物消费', '交通出行', '休闲娱乐', '其他支出']
sizes = [1200, 300, 200, 200, 100]
colors = ['#ff9999', '#99ff99', '#ffcc99', '#c2c2f0', '#ffb3e6']

# 面向对象风格
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(20, 7))

ax1.pie(sizes, labels=labels, colors=colors)
ax1.set_title('方式一：标准饼状图')

explode = (0.1, 0, 0, 0, 0)

```

```

ax2.pie(sizes, labels=labels, colors=colors, explode=explode)
ax2.set_title('方式二：百分比图')

ax3.pie(sizes, labels=labels, colors=colors, wedgeprops=dict(width=0.4))
ax3.set_title('方式三：环形图')

fig.suptitle('实验 2-4：十月份个人支出分析', fontsize=20)
plt.show()

```

5. 绘制散点图 $x = \sin t - t \cos t$, $y = \cos t - t \sin t$, t 区间为 $[-15, 15]$

```

import matplotlib.pyplot as plt

t_start=-15
t_end=15
t=np.linspace(t_start,t_end,500)

x = np.sin(t) - t * np.cos(t)
y = np.cos(t) - t * np.sin(t)

plt.figure(figsize=(10, 10))
plt.scatter(x, y, s=10, label=f't 范围: [{t_start}, {t_end}]')# 绘制散点图

plt.title('实验 2-5：散点图 ', fontsize=16)
plt.xlabel('X 轴:  $\sin(t) - t \cos(t)$ ', fontsize=12)
plt.ylabel('Y 轴:  $\cos(t) - t \sin(t)$ ', fontsize=12)

plt.grid(True, linestyle='-', alpha=0.7)

plt.legend()

```

```
plt.show()
```

6.绘制曲线图、阶梯图 $x = \sin t - t \cos t$, $y = \cos t - t \sin t$, t 区间为I

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
t_start=235127-15
```

```
t_end=235127+15
```

```
t=np.linspace(t_start,t_end,500)
```

```
x = np.sin(t) - t * np.cos(t)
```

```
y = np.cos(t) - t * np.sin(t)
```

```
plt.figure(figsize=(10, 10))
```

```
plt.plot(x, y, label=f't 范围: [{t_start}, {t_end}]', color='blue')# 绘制曲线图
```

```
plt.title('实验 2-6: 曲线图', fontsize=16)
```

```
plt.xlabel('X 轴:  $\sin(t) - t \cos(t)$ ', fontsize=12)
```

```
plt.ylabel('Y 轴:  $\cos(t) - t \sin(t)$ ', fontsize=12)
```

```
plt.grid(True, linestyle='-', alpha=0.7)
```

```
plt.legend()
```

```
plt.show()
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from matplotlib.ticker import ScalarFormatter
```

```
t_start=235127-15
```

```

t_end=235127+15
t=np.linspace(t_start,t_end,500)

x = np.sin(t) - t * np.cos(t)
y = np.cos(t) - t * np.sin(t)

plt.figure(figsize=(12, 8))
plt.step(t, y, label=f't 范围: [{t_start}, {t_end}]', color='red')# 绘制
阶梯图

plt.title('实验 2-6: 阶梯图 ', fontsize=16)
plt.xlabel('T 轴 (区间)', fontsize=12)
plt.ylabel('Y 轴:  $\cos(t) - t\sin(t)$ ', fontsize=12)

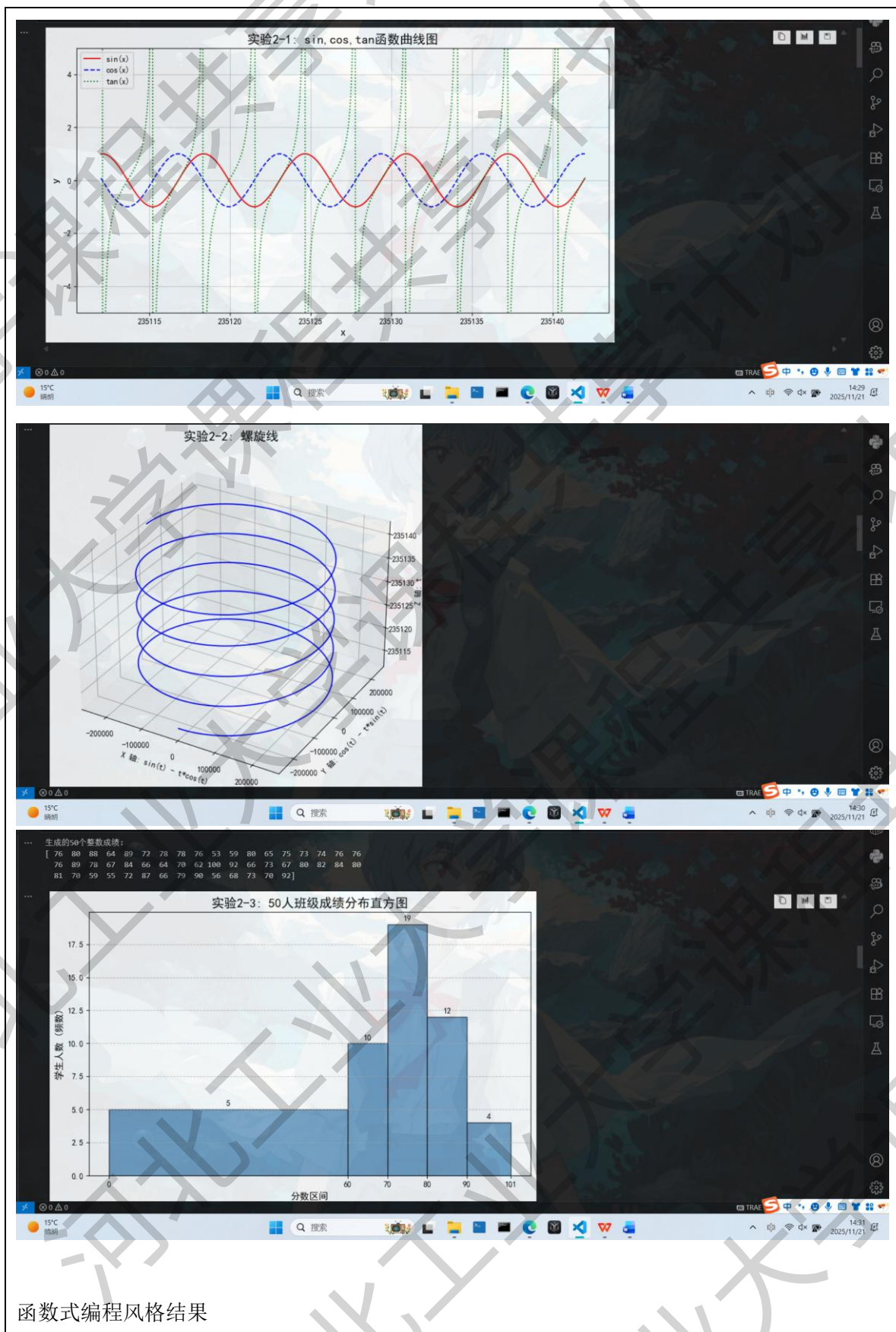
plt.grid(True, linestyle='-', alpha=0.7)

#这段为了避免科学计数法显示
ax = plt.gca()
formatter = ScalarFormatter(useOffset=False)
formatter.set_scientific(False)
ax.xaxis.set_major_formatter(formatter)

plt.legend()
plt.show()

```

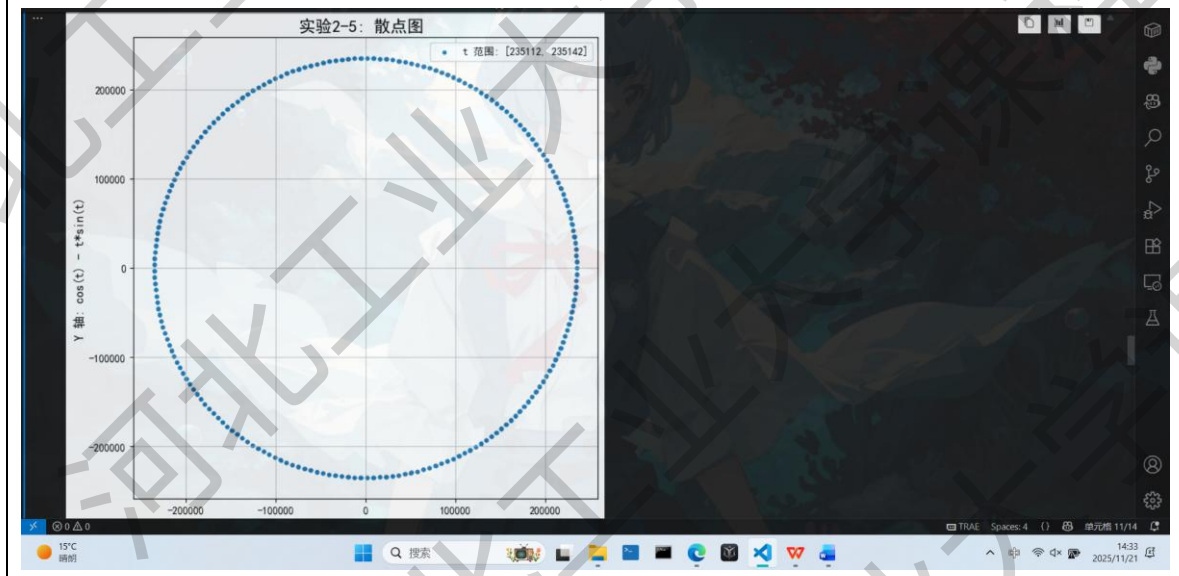
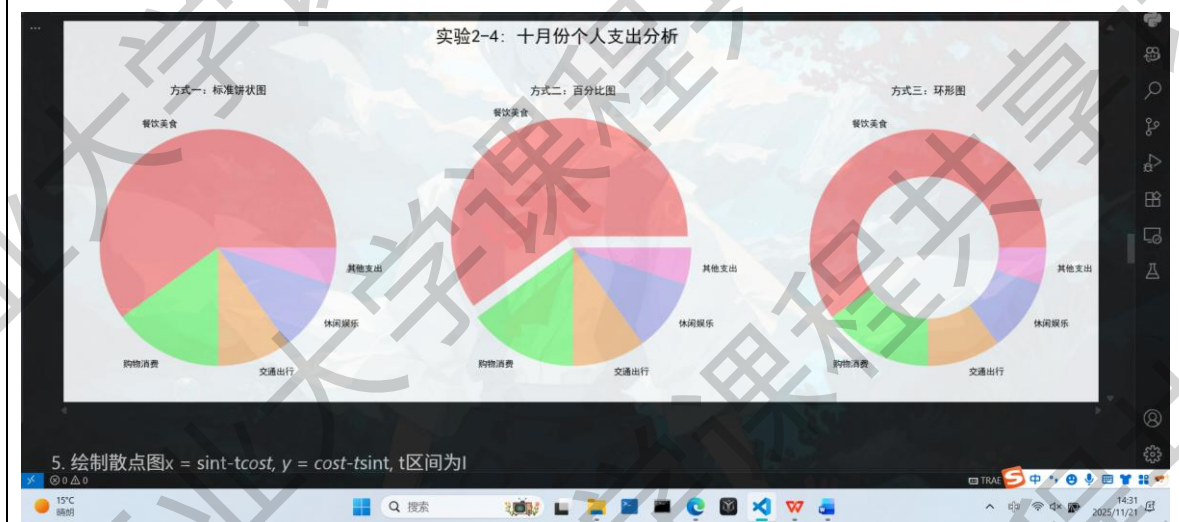
2. 运行结果（截图需同时截入右下角电脑时间）:

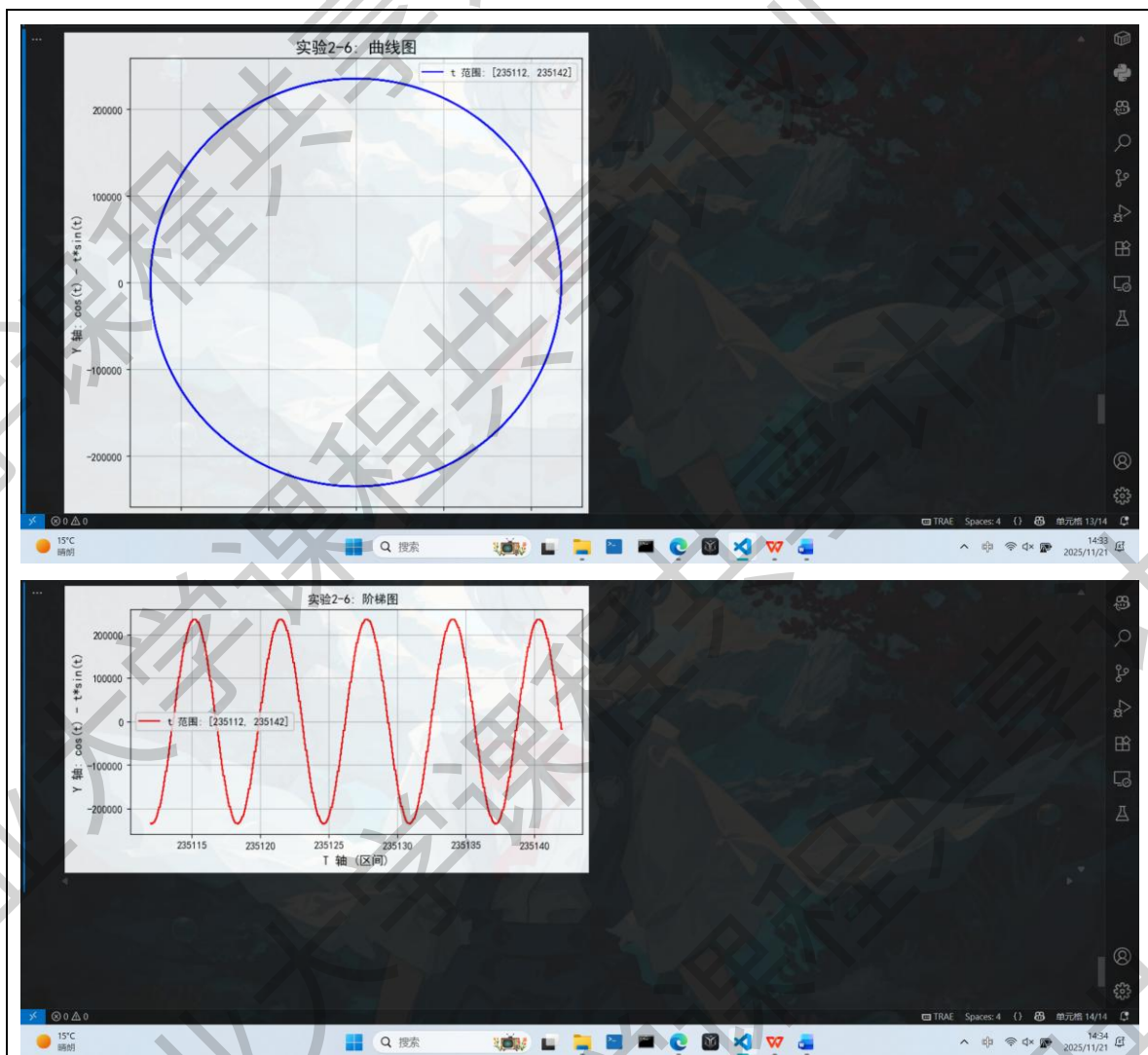


函数式编程风格结果



面向对象风格结果





三、实验的分析与思考（若有）

1. 对 Matplotlib 双重接口的理解（ax 与 plt）

在实验的初期，我遇到了关于 `ax.plot` 和 `plt.plot`，以及 `from ... import Axes3D` 的困惑。这引出了第一个深刻的思考：Matplotlib 存在两种绘图接口。一种是基于 `plt` 的“状态机”接口，适合快速、简单的绘图；另一种是基于 `ax`（坐标轴对象）的“面向对象”接口。在绘制三维图形或像饼图实验中那样需要并排（`plt.subplots(1, 3)`）绘制多个子图时，面向对象的 `ax` 接口是必须的，它让我能精确控制每一个独立的“画板”，而不是依赖 Matplotlib 自动寻找“当前”的图。

2. “合理”数据的生成与验证

在绘制成绩直方图时，我没有简单地使用均匀分布的随机数，而是探索了 `np.random.normal` 和 `np.random.randn`。这让我认识到，模拟真实世界的数据需要选择合适的统计模型。`grades = randn * std_dev + mean_score` 这一操作（平移和缩放）是统计学中从

标准正态分布构建任意正态分布的核心技巧。更重要的是，我使用了 `np.clip(grades, 0, 100)` 进行了数据“截断”。这反映了一个关键点：数学模型（正态分布）的理论范围是无限的，但现实世界的数据（如成绩）有其物理边界（0 到 100 分），我必须对模型生成的数据进行验证和清洗，使其符合现实逻辑。

3. 可视化的核心：控制“区间”与“范围”

在两个实验中我都遇到了“区间”问题。在直方图中，我通过传入一个列表 `bins = [0, 60, 70, 80, 90, 101]` 来自定义分桶，这比让 Matplotlib 自动均分（如 `bins=5`）要有意得多，因为它符合我对“不及格”、“良好”、“优秀”的社会认知。在绘制三角函数图时， $\tan(x)$ 的无穷大值导致 $\sin(x)$ 和 $\cos(x)$ 被“压扁”而无法观察。通过 `plt.ylim(-5, 5)` 手动设置 Y 轴范围，我“裁剪”了极端值，从而突出了我真正关心的信息。这说明，可视化不仅是“画出来”，更是通过主动控制范围和区间来“讲好故事”。

4. 调试“反常”的图形：偏移量与长宽比

在绘制三维螺旋线时，我遇到了最大的一个“陷阱”：X, Y 轴的坐标在 20 万量级，而 Z 轴（即 t ）虽然也是 20 万量级，但图形上只显示了 $[15, 40]$ 。这并不是代码错误，而是 Matplotlib 默认开启了坐标轴“偏移量”（Offset Notation）。这个“自作聪明”的功能在默认情况下造成了极大的误导。我必须通过底层的 `ScalarFormatter` 来强制禁用它。这个调试过程说明，不能盲目相信工具的默认设置，必须深入理解其背后的机制。同时，在绘制二维螺旋线散点图时，`plt.axis('equal')` 是必不可少的，它保证了 X, Y 轴的等比例缩放，防止了圆弧被错误地拉伸为椭圆。

5. 选择正确的图表类型

在最后的实验中，我对比了曲线图（`plot`）和阶梯图（`step`）。我发现，`plt.step(x, y)` 并不能画出我想要的图形，因为螺旋线的 x 坐标并不是单调递增的，这违反了阶梯图的绘图逻辑。正确的做法是绘制 `plt.step(t, y)`，因为 t 是严格递增的。这说明，选择图表类型不仅是“外观”的选择，更要理解该图表（如阶梯图）对数据的内在要求（如 X 轴的单调性）。

实验项目名称	实验三 常规数学统计运算		实验成绩
实 验 者	艾洋	专业班级	数据科学与大数据技术 数据 231
实验日期			2025 年 12 月 3 日

一、实验内容

1、随机生成一个 10x15 的高斯矩阵，均值为自己学号后两位，方差为 1。对该矩阵分别进行 LU、QR、奇异值，并展示分解结果。

2、利用软件求解优化问题：

$\min Z = -X_1 - 0.8X_2 - 1.2X_3,$

约束： $X_1 - X_2 + X_3 \leq 30,$

$3X_1 + 2X_2 + 4X_3 \leq 42,$

$3X_1 + 2X_2 \leq 30,$

$X_1, X_2, X_3 \geq 0.$

并找到其中有效约束。

3、对于函数 $f(x) = (x^2-5x+3)\exp(-4x)\cos x, x = 0: 0.1:1,$ 进行三种方式插值，并将插值曲线与原曲线绘制在同一个 Figure 窗口。

4、统计自己过去 12 个月实际生活花费的数值，并拟合成一条一次、二次、三次曲线，三条曲线分别用不同颜色和线型展示在同一张图里。

5、求解三重积分 $\int_{(-0.5, 1)} \int_{(0, 0.5)} \int_{(-0.5\pi, \pi)} (y \sin x \exp(x)+z \cos x x^2)dx dy dz.$

6、求微分方程 $xy' + y - \exp(x)=0,$ 在初始条件 $y(1)=2 \exp$ 下的特解，并画出解函数的图形。

7、某人进行射击，及每次命中的概率为 0.1，独立射击 50 次，求击中 10 次以上且 40 次以下的概率。

8、假设自己过去 12 个月实际生活花费的数值服从正态分布，请求出其均值和方差的极大似然估计。

二、实验源程序及结果截图

1. 源程序核心代码：

```
import numpy as np
import scipy.linalg as la

# --- 1. 设置参数 ---
```

```

mean = 27 # 均值
variance = 1# 方差
std_dev = np.sqrt(variance) # 标准差也为 1
rows = 10
cols = 15

A = np.random.normal(loc=mean, scale=std_dev, size=(rows, cols))
print(f"--- 原始矩阵 A (10x15) ---")
print(f"均值={mean}, 方差={variance}")
print(A)
print("-" * 100 + "\n")

print(f"--- LU 分解 (A = P.T @ L @ U) ---")
P_lu, L, U = la.lu(A)

print("P (置换矩阵, 10x10):")
print(P_lu)
print("\nL (下三角矩阵, 10x10):")
print(L)
print("\nU (上三角矩阵, 10x15):")
print(U)
print("-" * 100 + "\n")

print(f"--- QR 分解 (A = Q @ R) ---")
Q, R = la.qr(A, mode='full')

print("Q (正交矩阵, 10x10):")
print(Q)
print("\nR (上三角矩阵, 10x15):")

```

```

print(R)
print("--" * 100 + "\n")

print(f"---- 奇异值分解 SVD (A = U @ Sigma @ Vh) ----")
U_svd, s, Vh = la.svd(A, full_matrices=False) # 使用 "economy" SVD
print("U (左奇异向量矩阵, 10x10):")
print(U_svd)
print("\ns (奇异值, 10 个):")
print(s)
print("\nVh (V 的转置, 右奇异向量矩阵, 10x15):")
print(Vh)
print("--" * 100 + "\n")

import numpy as np
from scipy.optimize import linprog

# 目标函数: min Z = -X1 - 0.8*X2 - 1.2*X3
c = [-1, -0.8, -1.2]
#约束条件
A_ub = np.array([
    [1, -1, 1],
    [3, 2, 4],
    [3, 2, 0]
])
b_ub = np.array([30, 42, 30])
# 变量边界: X1, X2, X3 >= 0
x1_bounds = (0, None) # (0, None) 表示 0 <= X <= 无穷大
x2_bounds = (0, None)
x3_bounds = (0, None)

```

```

bounds = [x1_bounds, x2_bounds, x3_bounds]

# 求解
res = linprog(c, A_ub=A_ub, b_ub=b_ub, bounds=bounds, method='highs')

# 打印结果
print("---- 线性规划求解结果 ----")
print(res)
print("-" * 100 + "\n")

if res.success:
    print("求解成功!")
    print(f"最优目标函数值 (Min Z) = {res.fun:.4f}")
    print(f"最优解 (X1, X2, X3):")
    print(f"  X1 = {res.x[0]:.4f}")
    print(f"  X2 = {res.x[1]:.4f}")
    print(f"  X3 = {res.x[2]:.4f}")
    print("-" * 100 + "\n")

# 4. 寻找有效约束
print("---- 有效约束分析 ----")

constraint_names = [
    "约束 1:  $X1 - X2 + X3 \leq 30$ ",
    "约束 2:  $3X1 + 2X2 + 4X3 \leq 42$ ",
    "约束 3:  $3X1 + 2X2 \leq 30$ "
]

# 检查不等式约束

```

```

# res.slack 存储了松弛量 (b_ub - A_ub @ x)
# 松弛量接近 0 意味着约束是有效的
tolerance = 1e-9 # 浮点数比较的容差
effective_constraints = []
print("检查不等式约束:")
for i in range(len(b_ub)):
    slack = res.slack[i]
    print(f"    {constraint_names[i]}")
    print(f"    松弛量 (Slack) = {slack:.4f}")
    if slack < tolerance:
        print("    ==> 此约束为有效约束 (Binding)")
        effective_constraints.append(constraint_names[i])
    else:
        print("    ==> 此约束为无效约束 (Non-binding)")

# 检查变量的边界约束
print("\n检查变量边界约束 (>= 0):")
for i in range(len(res.x)):
    constraint_name = f"边界约束: X{i+1} >= 0"
    print(f"    {constraint_name}")
    print(f"    变量值 = {res.x[i]:.4f}")
    # 如果变量的值等于其下界 0
    if np.isclose(res.x[i], 0, atol=tolerance):
        print("    ==> 此约束为有效约束 (Binding)")
        effective_constraints.append(constraint_name)
    else:
        print("    ==> 此约束为无效约束 (Non-binding)")

print("\n--- 总结: 所有有效约束 ---")

```

```

    if not effective_constraints:
        print("没有找到有效约束。")
    else:
        for constr in effective_constraints:
            print(f" * {constr}")
else:
    print(f"求解失败: {res.message}")

import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import interp1d, BarycentricInterpolator
plt.rcParams['font.sans-serif'] = ['SimHei']
def f(x):
    return (x**2 - 5*x + 3) * np.exp(-4*x) * np.cos(x)
#插值节点, 步长 0.1
x_nodes = np.arange(0, 1.01, 0.1)
y_nodes = f(x_nodes)
#高密度取点
x_true = np.linspace(0, 1, 400)
y_true = f(x_true)

# --- 方法 1: 线性插值 ---
f_linear = interp1d(x_nodes, y_nodes, kind='linear')
y_linear = f_linear(x_true)

# --- 方法 2: 三次样条插值 ---
f_cubic = interp1d(x_nodes, y_nodes, kind='cubic')

```

```

y_cubic = f_cubic(x_true)

# --- 方法 3: 多项式插值 ---
f_poly = BarycentricInterpolator(x_nodes, y_nodes)
y_poly = f_poly(x_true)

# 5. 绘图
plt.figure(figsize=(8, 4))

# 绘制原始函数曲线
plt.plot(x_true, y_true, 'k-', linewidth=4, label='f(x) 原始曲线',
alpha=0.7)

# 绘制插值节点
plt.plot(x_nodes, y_nodes, 'ro', markersize=10, label='插值节点 (x, y)')

# 绘制三种插值曲线
plt.plot(x_true, y_linear, 'b--', linewidth=2, label='线性插值 (Linear)')
plt.plot(x_true, y_cubic, 'g-.', linewidth=2, label='三次样条插值 (Cubic Spline)')
plt.plot(x_true, y_poly, 'm:', linewidth=2, label='多项式插值 (Polynomial)')

# 添加图例、标题和网格
plt.title('插值方法对比', fontsize=12)
plt.xlabel('x', fontsize=8)
plt.ylabel('f(x)', fontsize=8)
plt.legend(fontsize=8)
plt.grid(True)
plt.show()

```

```

import numpy as np
import matplotlib.pyplot as plt

months = np.arange(1, 13)
expenses = np.array([
    2050, 2100, 1850, 2050, 2200, 2150,
    1900, 2250, 2000, 2100, 2050, 1950
])

x_smooth = np.linspace(months.min(), months.max(), 500)

# 一次
coeffs_1 = np.polyfit(months, expenses, 1)
y_linear = np.polyval(coeffs_1, x_smooth)

# 二次
coeffs_2 = np.polyfit(months, expenses, 2)
y_quadratic = np.polyval(coeffs_2, x_smooth)

# 三次
coeffs_3 = np.polyfit(months, expenses, 3)
y_cubic = np.polyval(coeffs_3, x_smooth)

plt.scatter(months, expenses, color='black', label='实际花费')

plt.plot(x_smooth, y_linear, 'r--', label='一次拟合')
plt.plot(x_smooth, y_quadratic, 'g-.', label='二次拟合')
plt.plot(x_smooth, y_cubic, 'b:', label='三次拟合')

```

```

plt.title('生活花费拟合曲线')
plt.xlabel('月份')
plt.ylabel('花费金额')
plt.xticks(months) # 让X轴显示1-12
plt.legend()
plt.grid(True)
plt.show()

import numpy as np
from scipy.integrate import tplquad

def function(z, y, x):
    return y * np.sin(x) * np.exp(x) + z * np.cos(x) * x**2

# 定义上下限
x_min, x_max = -np.pi/2, np.pi
y_min, y_max = 0, 0.5
z_min, z_max = -0.5, 1.0

result, error = tplquad(
    function,
    x_min, x_max,
    y_min, y_max,
    z_min, z_max
)

print(f"数值结果: {result}")

```

```

print(f"估算误差: {error}")

import sympy as sp

# 1. 定义符号
x = sp.symbols('x')
y = sp.Function('y')

# 2. 定义微分方程:  $xy' + y - \exp(x) = 0$ 
eq = sp.Eq(x * y(x).diff(x) + y(x) - sp.exp(x), 0)

# 3. 定义初始条件:  $y(1) = 2e$ 
# sp.E 代表常数 e
ics = {y(1): 2 * sp.E}

# 4. 求解特解
particular_solution = sp.dsolve(eq, ics=ics)

# 5. 打印解
print(f"--- 求解成功 ---")
print(f"特解为: {particular_solution}")

# 6. 绘制图形
sp.plot(particular_solution.rhs, (x, 0.1, 4),
        title="求解",
        xlabel='x',
        ylabel='y(x)')

```

```

from scipy.stats import binom

n = 50 # 射击次数
p = 0.1 # 命中概率

#  $P(10 < X < 40)$  等价于  $P(11 \leq X \leq 39)$ 
# 我们可以用  $P(X \leq 39) - P(X \leq 10)$  来计算

# binom.cdf(k, n, p) 计算  $X \leq k$  的概率
prob = binom.cdf(39, n, p) - binom.cdf(10, n, p)

print(f"试验次数 n={n}, 概率 p={p}")
print(f"击中 10 次以上且 40 次以下的概率  $P(11 \leq X \leq 39)$  为:")
print(f"{prob:.8f}")

import numpy as np

expenses = np.array([
    2050, 2100, 1850, 2050, 2200, 2150,
    1900, 2250, 2000, 2100, 2050, 1950
])

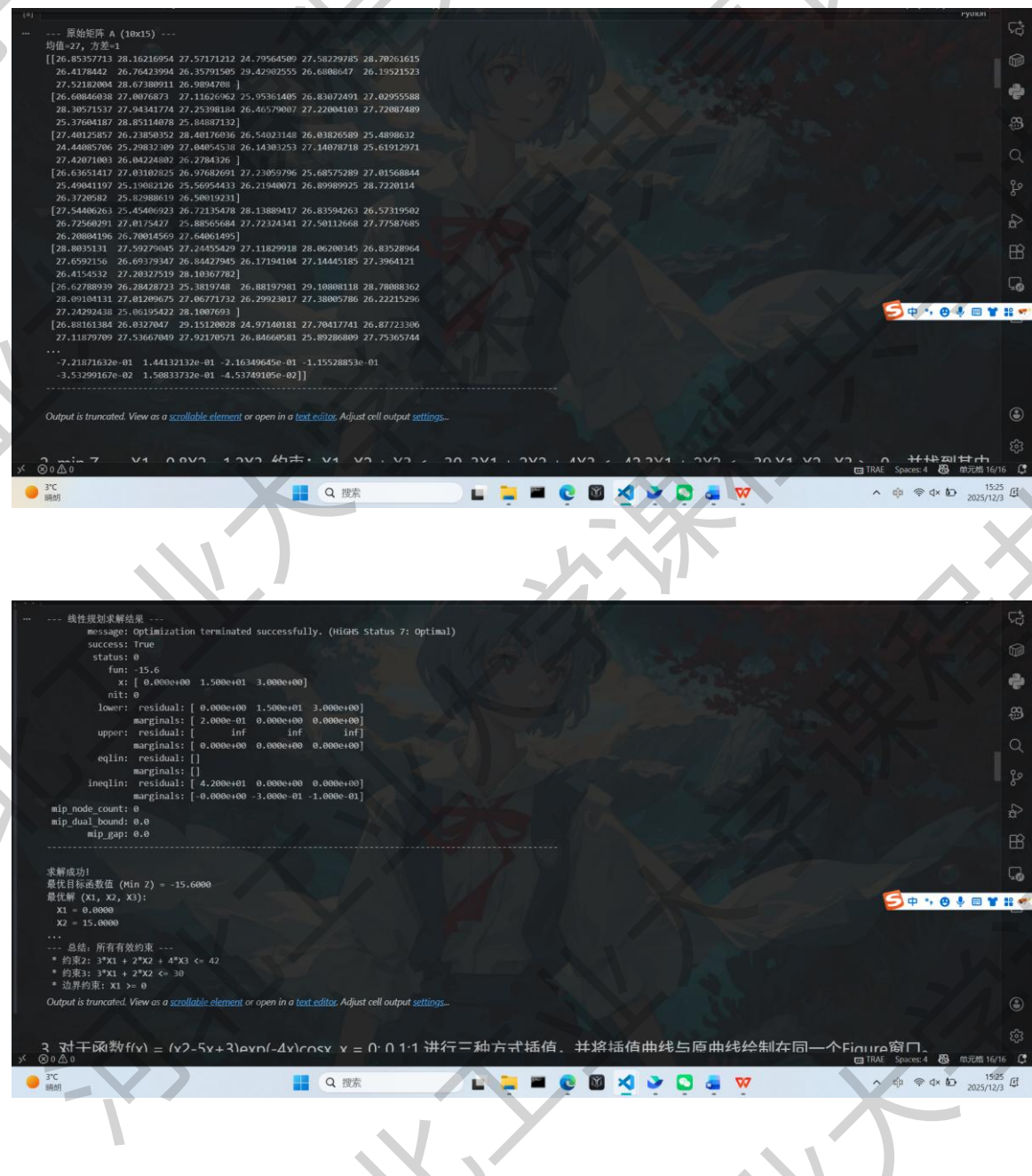
# 1. 均值的极大似然估计
mu_mle = np.mean(expenses)

# 2. 方差的极大似然估计 (np.var 默认 ddof=0, 即除以 n)
variance_mle = np.var(expenses)

```

```
print(f"数据 (n={len(expenses)}): \n{expenses}\n")
print(f"均值 ( $\mu$ ) 的极大似然估计: {mu_mle:.4f}")
print(f"方差 ( $\sigma^2$ ) 的极大似然估计: {variance_mle:.4f}")
```

2. 运行结果（截图需同时截入右下角电脑时间）：



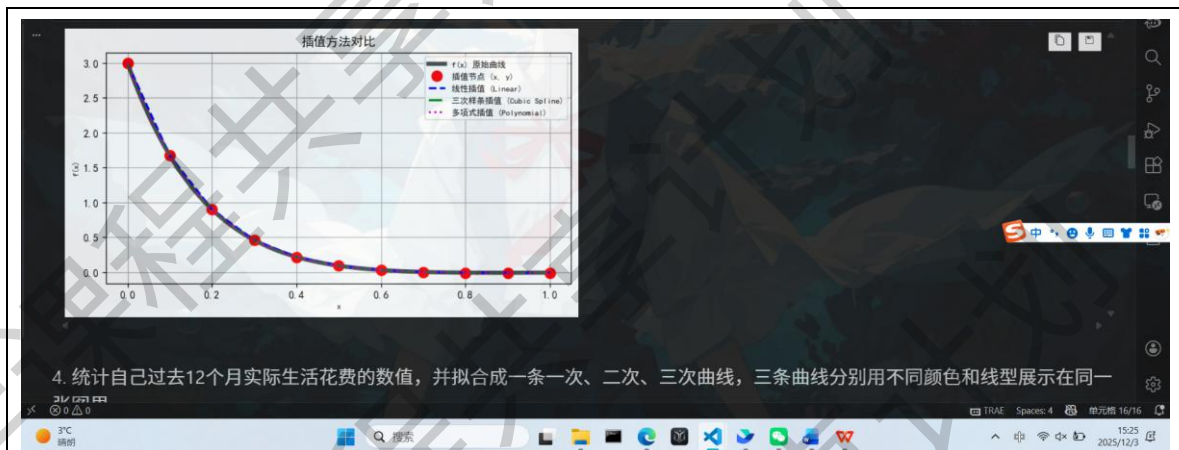
```
--- 原始矩阵 A (10x15) ---
均值-27, 方差-1
[[26.85357713 28.10210954 27.57171212 24.79564589 27.58229785 28.78261615
 26.4178442 26.76423994 26.35791505 29.42302555 26.6806647 26.19521523
 27.52182084 28.67380911 26.9894708 ]
 [26.60846038 27.0076873 27.11626962 25.95361405 26.83072491 27.02955588
 28.30571537 27.94341774 27.25398184 26.46579007 27.22004103 27.72087489
 25.37604187 28.85114078 25.84887132 ]
 [27.40125857 26.23850352 28.40176036 26.54023148 26.03826589 25.4898632
 24.44085706 25.29832309 27.04054538 26.14303253 27.14078718 25.61912971
 27.42071003 26.04224802 26.2784326 ]
 [26.63651417 27.03102425 26.97682691 27.23059796 25.68575289 27.01560844
 25.49041107 25.19002126 25.50954431 26.21940071 26.8990925 28.7220114
 26.3720582 25.82988619 26.50019231 ]
 [27.54406263 25.45406923 26.72135478 28.13889417 26.83594263 26.57319502
 26.77560291 27.0175427 25.88565684 27.72324341 27.50112668 27.77587685
 26.20804196 26.70014569 27.64061495 ]
 [28.8035131 27.59279045 27.24455429 27.11829918 28.06200345 26.83528964
 27.6592156 26.69379347 26.84427945 26.17194104 27.14445185 27.3964121
 26.4154532 27.20327519 28.10367782 ]
 [26.62788939 26.20420723 25.3819748 26.88197981 29.10080118 28.78088162
 28.00104131 27.01209075 27.00771732 26.29923017 27.38005786 26.22215296
 27.24202438 25.06105422 28.1007693 ]
 [26.88101384 26.0327047 29.15120028 24.97140181 27.70417741 26.87723306
 27.11879709 27.53667049 27.92170571 26.84660581 25.89286809 27.75365744
 ...
 -7.21071632e-01 1.44132132e-01 -2.16349645e-01 -1.15528853e-01
 -3.53299167e-02 1.50833732e-01 -4.53749105e-02]]

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings.

--- 线性规划求解结果 ---
message: Optimization terminated successfully. (HIGHS status 7: Optimal)
success: True
status: 0
fun: -15.6
nit: 0
lower: residual: [ 0.000e+00  1.500e+01  3.000e+00]
marginals: [ 2.000e-01  0.000e+00  0.000e+00]
upper: residual: [ inf inf inf]
marginals: [ 0.000e+00  0.000e+00  0.000e+00]
eqlin: residual: []
marginals: []
ineqlin: residual: [ 4.200e+01  0.000e+00  0.000e+00]
marginals: [ 0.000e+00 -3.000e-01 -1.000e-01]
mip_node_count: 0
mip_dual_bound: 0.0
mip_gap: 0.0

求解成功!
最优目标函数值 (Min Z) = -15.6000
最优解 (X1, X2, X3):
X1 = 0.0000
X2 = 15.0000
...
--- 总结: 所有有效约束 ---
* 约束2: 3*X1 + 2*X2 + 4*X3 <= 42
* 约束3: 3*X1 + 2*X2 <= 30
* 边界约束: X1 >= 0

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings.
```



```

import numpy as np
from scipy.integrate import tplquad

def function(z, y, x):
    return y * np.sin(x) * np.exp(x) + z * np.cos(x) * x**2

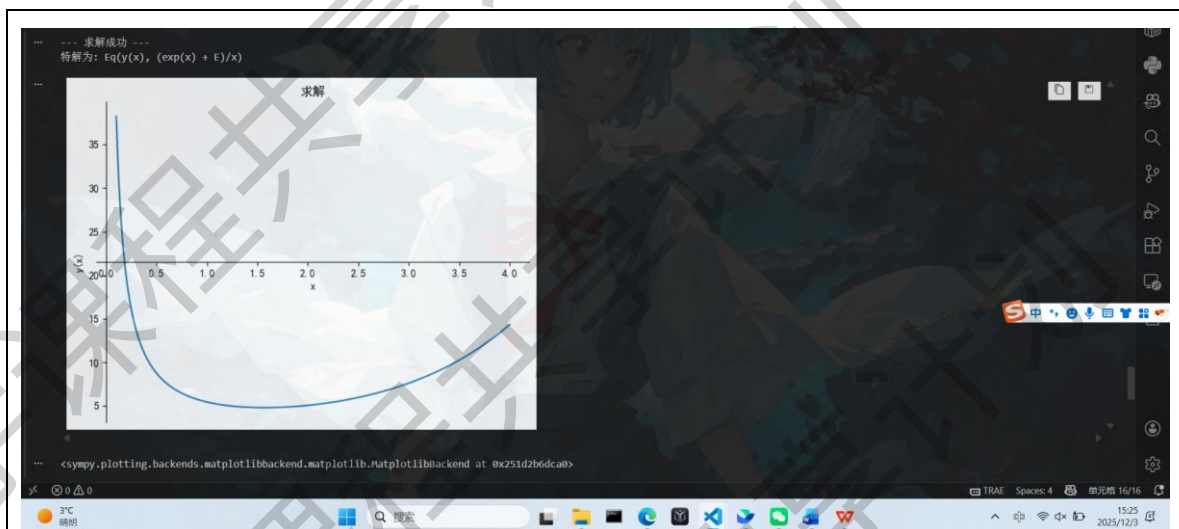
# 定义上下限
x_min, x_max = -np.pi/2, np.pi
y_min, y_max = 0, 0.5
z_min, z_max = -0.5, 1.0

result, error = tplquad(
    function,
    x_min, x_max,
    y_min, y_max,
    z_min, z_max
)

print(f"数值结果: {result}")
print(f"估算误差: {error}")

```

数值结果: 1.0984691058188318
估算误差: 6.007031615366822e-14



7. 某人进行射击，及每次命中的概率为0.1，独立射击50次，求击中10次以上且40次以下的概率。

```
from scipy.stats import binom

n = 50 # 射击次数
p = 0.1 # 命中概率

# P(10 < X < 40) 等价于 P(11 <= X <= 39)
# 我们可以用 P(X <= 39) - P(X <= 10) 来计算

# binom.cdf(k, n, p) 计算 X <= k 的概率
prob = binom.cdf(39, n, p) - binom.cdf(10, n, p)

print(f"试验次数 n={n}, 概率 p={p}")
print(f"击中10次以上且40次以下的概率 P(11 <= X <= 39) 为:")
print(f"{prob:.8f}")
```

试验次数 n=50, 概率 p=0.1
击中10次以上且40次以下的概率 P(11 <= X <= 39) 为:
0.00935460

8. 假设自己过去12个月实际生活花费的数值服从正态分布，请求出其均值和方差的极大似然估计。

```
import numpy as np # Ctrl+U 添加上下文对话
```

8. 假设自己过去12个月实际生活花费的数值服从正态分布，请求出其均值和方差的极大似然估计。

```
import numpy as np # Ctrl+U 添加上下文对话

expenses = np.array([
    2050, 2100, 1850, 2050, 2200, 2150,
    1900, 2250, 2000, 2100, 2050, 1950
])

# 1. 均值的极大似然估计
mu_mle = np.mean(expenses)

# 2. 方差的极大似然估计 (np.var 默认 ddof=0, 即除以 n)
variance_mle = np.var(expenses)

print(f"数据 (n={len(expenses)}): \n{expenses}\n")
print(f"均值 (μ) 的极大似然估计: {mu_mle:.4f}")
print(f"方差 (σ²) 的极大似然估计: {variance_mle:.4f}")
```

数据 (n=12):
[2050 2100 1850 2050 2200 2150 1900 2250 2000 2100 2050 1950]

均值 (μ) 的极大似然估计: 2054.1667
方差 (σ²) 的极大似然估计: 12690.9722

三、实验的分析与思考（若有）

为正确的问题选择正确的工具。例如，求解微分方程时，SymPy 提供了 $y(x) = (e + e^x)/x$ 这样的**符号解**，让我们能理解其数学本质；而 Scipy 则提供了**数值解**，用于高效计算结果。同样，在矩阵分解中，LU 专用于解方程，而 SVD 则是数据降维（如 PCA）和推荐系统的基石，因为它的奇异值揭示了“数据的重要性”。

区分“拟合”与“插值”：如何看待数据

我们对两组数据做了不同处理：

- **插值（实验 3）**：假设数据是**完美的**，目标是找到穿过所有点的平滑曲线。但实验也警示了高阶多项式插值会在边缘剧烈振荡（龙格现象），稳定性差。
- **拟合（实验 4）**：假设数据是**带噪声的**，目标是找到代表“总体趋势”的曲线，不强求穿过任何一点。这在处理真实花费数据时更合理，能避免“过拟合”。

实验项目名称	实验四 机器学习算法建模与求解		实验成绩	
实 验 者	艾洋	专业班级	数据科学与大数据技术 数据 231	
实验日期			2025 年 12 月 3 日	

一、实验内容

1、对于下表中的数据，对 1990 年-2005 年内的数据建立人口自然增长率对于国民总收入、CPI 增长率和人均 GDP 的三元线性回归模型。

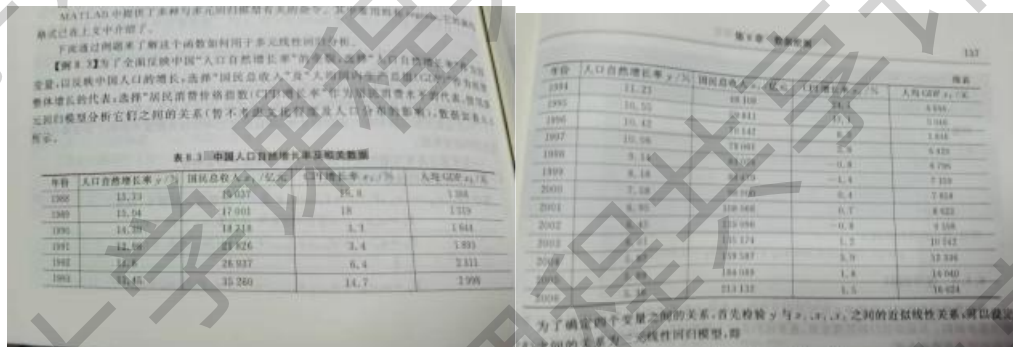


表 8-3 中国人口自然增长率及宏观经济数据

年份	人口自然增长率 y (%)	国民总收入 x_1 (亿元)	CPI 增长率 x_2 (%)	人均 GDP x_3 (元)
1990	13.23	19037	3.8	1388
1991	13.04	17001	1.8	1239
1992	14.39	18248	3.7	1364
1993	12.58	20926	3.4	1593
1994	12.8	26937	6.4	2133
1995	13.45	35240	14.7	2999

表 8-4 中国人口自然增长率及宏观经济数据 (续)

年份	人口自然增长率 y (%)	国民总收入 x_1 (亿元)	CPI 增长率 x_2 (%)	人均 GDP x_3 (元)
1996	11.21	48109	24.1	4599
1997	10.55	58411	12.1	5566
1998	10.42	70142	12.1	6566
1999	10.08	78083	8.8	7439
2000	9.14	93028	0.8	8796
2001	8.18	10495	-1.4	9739
2002	7.58	120465	0.4	10808
2003	6.93	139566	0.7	12023
2004	6.42	159586	-1.8	13588
2005	5.91	173174	1.2	15142
2006	5.49	193587	3.9	16938
2007	5.06	224089	1.8	18940
2008	4.64	253135	5.5	21024

2、下载 UCI 中 wine 数据集：<http://archive.ics.uci.edu/ml/datasets/Wine>。所下载数据可以用 txt 打开，其中每一行数据为一种 Wine 的记录，每条记录包含 14 个维度，其中第一维为该 Wine 类别，后面 13 维为具体的 Wine 属性。请基于所有的 178 个 Wine 样本对 Wine 的 13 个维度进行 PCA 降维分析，将贡献率之和大于 90%的成分提取，并将原 13 维属性数据映射为新数据。

3、

(1) 随机生成均值、方差各不相同，且相互之间有少量交叉的 3 个类，每类 30 个样本，用不同的颜色进行展示。

(2) 通过 keamns 聚类分析，将所有的数据分成 3 类、4 类、5 类，每一类用不同颜色展示。共形成 4 张图。

4、随机生成完全不交叉的 2 个类，每个类包含 30 个样本，用 SVM 进行分类和返回所有支撑向量，并以合适方式进行 Figure 展示。

二、实验源程序及结果截图

1. 源程序核心代码：

```
import pandas as pd
import statsmodels.api as sm
```

```

import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['SimHei']

# 1. 整理数据 (1990-2005 年)
data = {
    'Year': [1990, 1991, 1992, 1993, 1994, 1995, 1996, 1997, 1998, 1999,
             2000, 2001, 2002, 2003, 2004, 2005],
    # y: 人口自然增长率 (%)
    'y_PopGrowth': [14.39, 12.98, 11.6, 11.45, 11.21, 10.55, 10.42,
                    10.06, 9.14, 8.18, 7.58, 6.95, 6.45, 6.01, 5.87, 5.89],
    # x1: 国民总收入 (亿元)
    'x1_GNI': [18718, 21826, 26937, 35260, 48108, 59811, 70142, 78061,
              83024, 88479, 98000, 108048, 119086, 135174, 159587, 184089],
    # x2: CPI 增长率 (%)
    'x2_CPI': [3.1, 3.4, 6.4, 14.7, 24.1, 17.1, 8.3, 2.8, -0.8, -1.4,
              0.4, 0.7, -0.8, 1.2, 3.9, 1.8],
    # x3: 人均 GDP (元)
    'x3_GDP_per_capita': [1644, 1893, 2311, 2998, 4044, 4846, 5846,
                          6420, 6796, 7158, 7858, 8622, 9358, 10542, 12336, 14040]
}

# 转换为 DataFrame
df = pd.DataFrame(data)

# 2. 准备变量
# Y 是因变量
y = df['y_PopGrowth']
# X 是自变量
X = df[['x1_GNI', 'x2_CPI', 'x3_GDP_per_capita']]

```

```

# 3. 添加常数项 (截距)
# statsmodels.OLS 默认不包含截距项, 我们用 add_constant 手动添加
X_with_const = sm.add_constant(X)

# 4. 建立并拟合 OLS (Ordinary Least Squares) 模型
model = sm.OLS(y, X_with_const)
results = model.fit()

# 5. 打印回归结果摘要
print("三元线性回归模型结果摘要")
print(results.summary())

import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# --- 1. 加载数据 ---

file_path =
r'D:\\autumn\\CS_Experiment\\DataAnalysis_Experiment\\wine\\wine.data'

try:
    # UCI 数据集通常没有表头 (header=None)
    df = pd.read_csv(file_path, header=None)
    print(f"文件 '{file_path}' 读取成功!")

```

```

except FileNotFoundError:
    print(f"错误：未找到文件 '{file_path}'。")
    print("请再次检查路径和文件名是否完全正确。")
except Exception as e:
    print(f"发生其他错误：{e}")

# --- 2. 分离特征 (X) 和标签 (y) ---

# 第 1 维 (索引 0) 是类别标签
y = df[0]

# 后面 13 维 (索引 1 到 13) 是属性特征
X = df.loc[:, 1:]

# --- 3. 标准化数据 ---

# PCA 对数据的尺度非常敏感 (即方差大小)
# 在执行 PCA 之前, 必须对特征进行标准化 (均值为 0, 方差为 1)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# --- 4. 执行 PCA 分析 ---

# 关键步骤: 设置 n_components=0.9
# sklearn 将自动选择能解释 90% (0.9) 以上方差所需的最少主成分数量
print("正在执行 PCA, 目标累计贡献率 > 90% ...")
pca = PCA(n_components=0.9)

# 拟合数据并进行转换 (映射)

```

```

X_new = pca.fit_transform(X_scaled)

# --- 5. 打印分析结果 ---

print("\n--- PCA 降维分析结果 ---")
print(f"原始数据维度: {X.shape}")
print(f"标准化后数据维度: {X_scaled.shape}")
print(f"降维后新数据维度: {X_new.shape}")

print("-----")
# pca.n_components_ 会告诉我们它自动选择了多少个成分
print(f"自动选择的主成分数量: {pca.n_components_}")

# pca.explained_variance_ratio_ 存储了每个主成分的贡献率
print(f"各主成分的贡献率: {pca.explained_variance_ratio_}")

# 计算并打印累计贡献率
total_variance = np.sum(pca.explained_variance_ratio_)
print(f"    累    计    贡    献    率    :    {total_variance:.4f}    (    即
{total_variance*100:.2f}%)")

print("\n--- 映射后的新数据 (前 5 行) ---")
# 设置 numpy 打印选项以便于阅读
np.set_printoptions(precision=4, suppress=True)
print(X_new[:5])

import numpy as np
import matplotlib.pyplot as plt

```

```

from sklearn.cluster import KMeans

# --- (1) 生成数据 ---

# 为了让结果可复现，设置一个随机种子
np.random.seed(42)

n_samples = 30 # 每类的样本数

# 定义 3 个类的均值和协方差矩阵
# 均值不同，协方差矩阵（控制方差和形状）也不同
mean1 = [0, 0]
cov1 = [[1, 0.5], [0.5, 1]] # 倾斜的椭圆

mean2 = [4, 5]
cov2 = [[1.5, -0.7], [-0.7, 1.5]] # 另一个方向倾斜的椭圆

mean3 = [1, 7]
cov3 = [[0.5, 0], [0, 2]] # 窄高的椭圆

# 生成数据
class1 = np.random.multivariate_normal(mean1, cov1, n_samples)
class2 = np.random.multivariate_normal(mean2, cov2, n_samples)
class3 = np.random.multivariate_normal(mean3, cov3, n_samples)

# 将所有数据合并到一个数组中，用于后续聚类
# X 是一个 (90, 2) 的数组
X = np.vstack((class1, class2, class3))

```

```

# --- 可视化 (1): 展示原始的 3 个类 ---
plt.figure(figsize=(8, 6))
plt.scatter(class1[:, 0], class1[:, 1], c='r', label='原始类别 1')
plt.scatter(class2[:, 0], class2[:, 1], c='g', label='原始类别 2')
plt.scatter(class3[:, 0], class3[:, 1], c='b', label='原始类别 3')
plt.title('(1) 原始生成的数据 (Ground Truth)')
plt.xlabel('特征 1')
plt.ylabel('特征 2')
plt.legend()
plt.grid(True)
plt.show()

# ---- (2) K-Means 聚类分析 ----

# 我们要分析的 K 值
k_values = [3, 4, 5]

# 创建一个 1x3 的子图网格，用于并排显示 K=3, 4, 5 的结果
# (figsize 可以根据您的屏幕调整)
fig, axes = plt.subplots(1, len(k_values), figsize=(18, 5))

print(f"\n--- 正在执行 K-Means 聚类 (K={k_values}) ----")

for i, k in enumerate(k_values):
    # 创建 KMeans 模型
    # n_init=10 是为了防止陷入局部最优解，'auto' 是新版本的默认值
    kmeans = KMeans(n_clusters=k, random_state=42, n_init='auto')

    # 拟合数据并预测标签

```

```

labels = kmeans.fit_predict(X)

# 获取聚类中心
centers = kmeans.cluster_centers_

# --- 可视化 (2): 在对应的子图上绘图 ---
ax = axes[i]

# 使用 labels 作为颜色, cmap='viridis' 是一种常用的颜色映射
scatter = ax.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis',
alpha=0.7)

# 标出聚类中心 (用红色大 'X')
ax.scatter(centers[:, 0], centers[:, 1], c='red', marker='X', s=200,
label='聚类中心')

ax.set_title(f' (2) K-Means 聚类 (K={k}) ')
ax.set_xlabel('特征 1')
ax.set_ylabel('特征 2')
ax.legend()
ax.grid(True)

# 调整布局并显示 K-Means 结果图
plt.tight_layout()
plt.show()

import numpy as np
import matplotlib.pyplot as plt
from sklearn import svm

```

```

from sklearn.datasets import make_blobs

# --- (1) 生成完全不交叉的 2 个类 ---

# n_samples=[30, 30] 确保每类 30 个样本
# centers=[[0, 0], [6, 6]] 两个中心点相距很远
# cluster_std=0.8 控制簇内方差，确保不重叠
# random_state=0 保证每次运行结果一致
X, y = make_blobs(n_samples=[30, 30],
                  centers=[[0, 0], [6, 6]],
                  cluster_std=0.8,
                  random_state=0)

# --- (2) 训练 SVM 模型 ---

# 我们使用 C=1000（一个很大的值）来强制执行“硬间隔”（Hard Margin）
# 因为我们知道数据是完全可分的
clf = svm.SVC(kernel='linear', C=1000)
clf.fit(X, y)

# --- (3) 返回所有支撑向量 ---

# 从拟合后的模型中获取支撑向量
support_vectors = clf.support_vectors_

print(f"--- SVM 支撑向量（共 {len(support_vectors)} 个） ---")
print(support_vectors)

# --- (4) 以合适方式进行 Figure 展示 ---

```

```

plt.figure(figsize=(10, 7))

# 1. 绘制所有数据点，用颜色区分
# c=y 表示颜色根据标签 y (0 或 1) 来决定
# cmap=plt.cm.Paired 是一种清晰的颜色方案
plt.scatter(X[:, 0], X[:, 1], c=y, s=40, cmap=plt.cm.Paired)

# 2. 绘制支撑向量
# 支撑向量被一个大的、黑边的、无填充的圆圈高亮显示
plt.scatter(support_vectors[:, 0], support_vectors[:, 1],
            s=120, facecolors='none', edgecolors='k',
            linewidths=1.5, label='Support Vectors')

# 3. 绘制决策边界和间隔 (Margins)
# 这是绘制 SVM 边界的标准方法
ax = plt.gca() # 获取当前绘图轴
xlim = ax.get_xlim()
ylim = ax.get_ylim()

# 创建网格来评估模型
xx = np.linspace(xlim[0], xlim[1], 30)
yy = np.linspace(ylim[0], ylim[1], 30)
YY, XX = np.meshgrid(yy, xx)
xy = np.vstack([XX.ravel(), YY.ravel()]).T

# decision_function() 返回每个点到决策边界的“距离”
Z = clf.decision_function(xy).reshape(XX.shape)

```

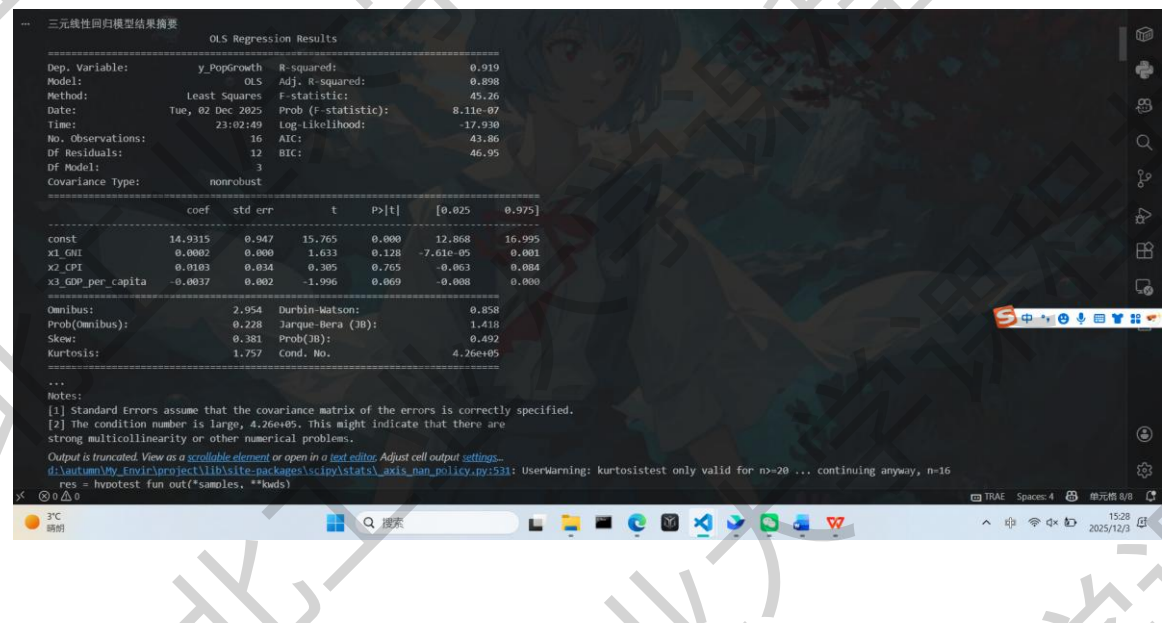
```

# 绘制决策边界 (Z=0) 和间隔 (Z=-1, Z=1)
# levels=[-1, 0, 1] 分别对应 间隔、决策边界、间隔
ax.contour(XX, YY, Z, colors='k', levels=[-1, 0, 1], alpha=0.5,
           linestyles=['--', '-', '--'])

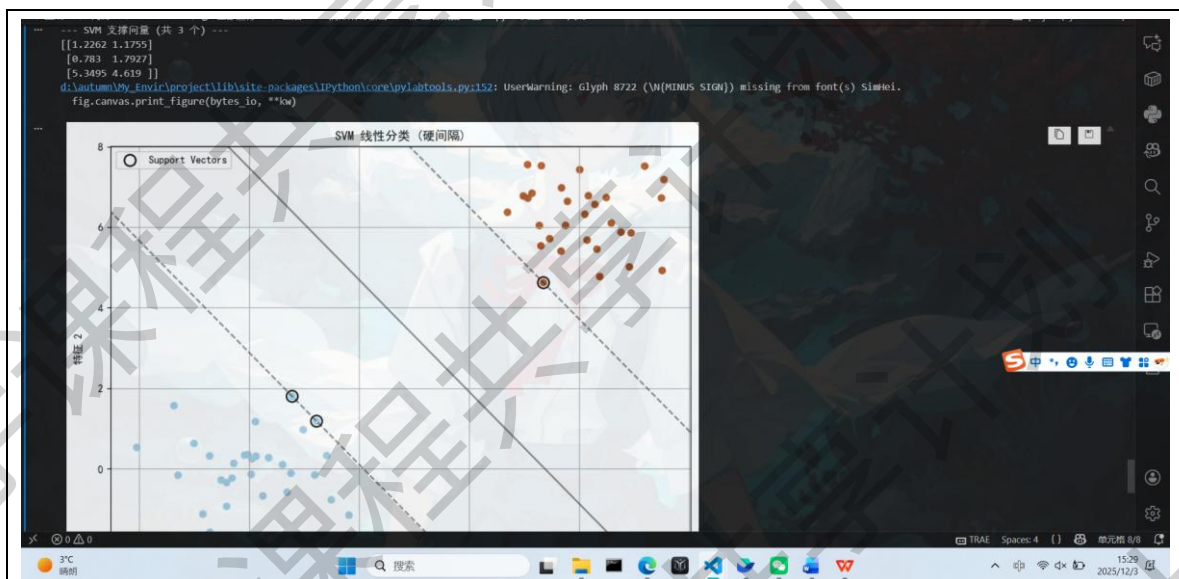
# --- 图表美化 ---
plt.title('SVM 线性分类 (硬间隔)')
plt.xlabel('特征 1')
plt.ylabel('特征 2')
plt.legend()
plt.grid(True)
plt.show()

```

2. 运行结果（截图需同时截入右下角电脑时间）：







三、实验的分析与思考（若有）

本次实验通过多元线性回归模型对 1990 年至 2005 年间的经济数据进行了深入分析，探究了国民总收入、CPI 增长率及人均 GDP 对人口自然增长率的影响。实验结果表明，线性回归能够有效量化多个自变量与因变量之间的关联程度，揭示宏观经济指标与人口变动之间的潜在规律。同时，在针对 Wine 数据集的处理中，利用 PCA（主成分分析）技术成功将 13 维的复杂属性降维至贡献率之和大于 90% 的主成分，这不仅极大地降低了数据的计算复杂度，还有效保留了原始数据中的关键特征信息，验证了降维技术在高维数据预处理中的核心作用。

在机器学习算法的对比实验中，K-means 聚类与 SVM 支持向量机的表现各具特色。通过对随机生成的交叉样本进行 K-means 聚类分析，观察 k 值取 3、4、5 时的不同聚类效果，直观体会到了无监督学习中超参数选择对模型结果的显著影响，以及算法处理边界模糊数据时的局限性。而在非交叉样本的 SVM 分类实验中，通过可视化展示支撑向量和决策边界，清晰地验证了 SVM 寻找最大间隔超平面的能力，证明了其在处理线性可分数据时具有极高的鲁棒性和准确性。

实验项目名称	实验五 分析算法进阶		实验成绩	
实 验 者	艾洋	专业班级	数据科学与大数据技术 数据 231	
实验日期			2025 年 12 月 24 日	

一、实验内容

1、分类任务

下表为城市中空气各气体成分含量与污染分类的情况。任选两种分类算法对待分类样本实现分类。

表 8.7 空气各气体成分含量与污染分类的情况						
氯	硫化氢	二氧化硫	碳四	环氧氯丙烷	环己烷	污染类别
0.056 0	0.084 0	0.031 0	0.038 0	0.008 1	0.022 0	1
0.040 0	0.055 0	0.100 0	0.110 0	0.022 0	0.007 3	1
0.050 0	0.074 0	0.041 0	0.048 0	0.007 1	0.020 0	1
0.045 0	0.050 0	0.110 0	0.100 0	0.025 0	0.006 3	1
0.038 0	0.130 0	0.079 0	0.170 0	0.058 0	0.043 0	2
0.030 0	0.110 0	0.070 0	0.160 0	0.050 0	0.046 0	2
0.034 0	0.095 0	0.058 0	0.160 0	0.200 0	0.029 0	1
0.030 0	0.090 0	0.068 0	0.180 0	0.220 0	0.039 0	1
0.084 0	0.066 0	0.029 0	0.320 0	0.012 0	0.041 0	2
0.085 0	0.076 0	0.019 0	0.300 0	0.010 0	0.040 0	2
0.064 0	0.072 0	0.020 0	0.250 0	0.028 0	0.038 0	2
0.054 0	0.065 0	0.022 0	0.280 0	0.021 0	0.040 0	2
0.048 0	0.089 0	0.062 0	0.260 0	0.038 0	0.036 0	2
0.045 0	0.092 0	0.072 0	0.200 0	0.035 0	0.032 0	2
0.069 0	0.087 0	0.027 0	0.050 0	0.089 0	0.021 0	1

表 8.8 待判断样本						
氯	硫化氢	二氧化硫	碳四	环氧氯丙烷	环己烷	污染类别
0.052 0	0.084 0	0.021 0	0.037 0	0.007 1	0.022 0	
0.041 0	0.055 0	0.110 0	0.110 0	0.021 0	0.007 3	
0.030 0	0.112 0	0.072 0	0.160 0	0.056 0	0.021 0	
0.074 0	0.083 0	0.105 0	0.190 0	0.020 0	1.000 0	

2、聚类任务

为了研究世界各国森林、草原资源的分布规律，共抽取了 21 个国家的数据，每个国家 4 项指标，数据如下表所示。使用该数据对国别进行聚类分析。

编号	国别	森林面积 /万公顷	森林覆盖率 /%	林木蓄积量 /亿立方米	草原面积 /万公顷
1	中国	11 978	12.5	93.5	31 908
2	美国	28 446	30.4	202.0	23 754
3	日本	2 501	67.2	24.8	58
4	德国	1 028	28.4	14.0	599
5	英国	210	8.6	1.5	1 147
6	法国	1458	26.7	16.0	1 288
7	意大利	635	21.1	3.6	514
8	加拿大	32 613	32.7	192.8	2 385
9	澳大利亚	10 700	13.9	10.5	45 190
10	俄罗斯	92 000	41.1	841.5	37 370
11	捷克	458	35.8	8.9	168
12	波兰	868	27.8	11.4	405
13	匈牙利	161	17.4	2.5	129
14	南斯拉夫*	929	36.3	11.4	640
15	罗马尼亚	634	26.7	11.3	447
16	保加利亚	385	34.7	2.5	200
17	印度	6 748	20.5	29.0	1 200
18	印度尼西亚	2 180	84.0	33.7	1 200
19	尼日利亚	1 490	16.1	0.8	2 090
20	墨西哥	4 850	24.6	32.6	7 450
21	巴西	57 500	67.6	238.0	15 900

★ 本表是历史数据样本，1992 年该国已分解为几个国家

3、关联规则

从以下网址下载 UCI 中的 adult 数据 <https://archive.ics.uci.edu/ml/datasets/Adult>

首先，抽取一年收入大于 50k 的数据，针对工作环境、教育、婚姻状况、职业、关系、种族和性别，7 个类别变量，运用关联规则分析找出属性变量中哪些变量间有关系？（提取关联性排名前 10 的关联规则）

二、实验源程序及结果截图

1. 源程序核心代码：

```
import numpy as np
import pandas as pd

from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
```

```

from sklearn.preprocessing import StandardScaler

# =====
# 1. 数据录入 (Data Entry)
# =====

# 训练集特征 (X_train) - 来自表 8.7
# 顺序: 氯, 硫化氢, 二氧化硫, 碳四, 环氧氯丙烷, 环己烷
X_train = np.array([
    [0.0560, 0.0840, 0.0310, 0.0380, 0.0081, 0.0220], # 样本 1
    [0.0400, 0.0550, 0.1000, 0.1100, 0.0220, 0.0073], # 样本 2
    [0.0500, 0.0740, 0.0410, 0.0480, 0.0071, 0.0200], # 样本 3
    [0.0450, 0.0500, 0.1100, 0.1000, 0.0250, 0.0063], # 样本 4
    [0.0380, 0.1300, 0.0790, 0.1700, 0.0580, 0.0430], # 样本 5
    [0.0300, 0.1100, 0.0700, 0.1600, 0.0500, 0.0460], # 样本 6
    [0.0340, 0.0950, 0.0580, 0.1600, 0.2000, 0.0290], # 样本 7
    [0.0300, 0.0900, 0.0680, 0.1800, 0.2200, 0.0390], # 样本 8
    [0.0840, 0.0660, 0.0290, 0.3200, 0.0120, 0.0410], # 样本 9
    [0.0850, 0.0760, 0.0190, 0.3000, 0.0100, 0.0400], # 样本 10
    [0.0640, 0.0720, 0.0200, 0.2500, 0.0280, 0.0380], # 样本 11
    [0.0540, 0.0650, 0.0220, 0.2800, 0.0210, 0.0400], # 样本 12
    [0.0480, 0.0890, 0.0620, 0.2600, 0.0380, 0.0360], # 样本 13
    [0.0450, 0.0920, 0.0720, 0.2000, 0.0350, 0.0320], # 样本 14
    [0.0690, 0.0870, 0.0270, 0.0500, 0.0890, 0.0210] # 样本 15
])

# 训练集标签 (y_train) - 来自表 8.7 (污染类别)
y_train = np.array([1, 1, 1, 1, 2, 2, 1, 1, 2, 2, 2, 2, 2, 2, 1])

```

```

# 待判断样本 (X_test) - 来自表 8.8
X_test = np.array([
    [0.0520, 0.0840, 0.0210, 0.0370, 0.0071, 0.0220], # 待判 1
    [0.0410, 0.0550, 0.1100, 0.1100, 0.0210, 0.0073], # 待判 2
    [0.0300, 0.1120, 0.0720, 0.1600, 0.0560, 0.0210], # 待判 3
    [0.0740, 0.0830, 0.1050, 0.1900, 0.0200, 1.0000] # 待判 4
])

# =====
# 2. 模型训练与预测 (Training & Prediction)
# =====

# --- 预处理：标准化 (Standardization) ---
# KNN 算法基于距离，特征值的量纲差异会影响结果，因此需要标准化
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# --- 算法 1: K-近邻 (KNN) ---
# 选择最近的 3 个邻居进行投票
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train_scaled, y_train)
y_pred_knn = knn.predict(X_test_scaled)

# --- 算法 2: 决策树 (Decision Tree) ---
# 决策树通常不需要数据标准化
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)
y_pred_dt = dt.predict(X_test)

```

```

# =====
# 3. 输出结果 (Output)
# =====
print("实验结果：")
print("-" * 50)
print(f"{'样本 ID':<8} | {'KNN 预测类别':<15} | {'决策树 预测类别':<15}")
print("-" * 50)

for i in range(len(X_test)):
    print(f"样本 {i+1:<5} | {y_pred_knn[i]:<19} | {y_pred_dt[i]:<19}")
print("-" * 50)

df_result = pd.DataFrame({
    '待判样本': [1, 2, 3, 4],
    'KNN 结果': y_pred_knn,
    '决策树结果': y_pred_dt
})

# print("\n 数据表格形式：")
# print(df_result)

二

import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

# =====

```

```

# 1. 数据录入 (Data Entry)
# =====
# 将图片中的表格数据转化为 DataFrame
data = {
    'Country': [
        '中国', '美国', '日本', '德国', '英国', '法国', '意大利',
        '加拿大', '澳大利亚', '俄罗斯', '捷克', '波兰', '匈牙利',
        '南斯拉夫', '罗马尼亚', '保加利亚', '印度', '印度尼西亚',
        '尼日利亚', '墨西哥', '巴西'
    ],
    '森林面积': [
        11978, 28446, 2501, 1028, 210, 1458, 635,
        32613, 10700, 92000, 458, 868, 161,
        929, 634, 385, 6748, 2180,
        1490, 4850, 57500
    ],
    '森林覆盖率': [
        12.5, 30.4, 67.2, 28.4, 8.6, 26.7, 21.1,
        32.7, 13.9, 41.1, 35.8, 27.8, 17.4,
        36.3, 26.7, 34.7, 20.5, 84.0,
        16.1, 24.6, 67.6
    ],
    '林木蓄积量': [
        93.5, 202.0, 24.8, 14.0, 1.5, 16.0, 3.6,
        192.8, 10.5, 841.5, 8.9, 11.4, 2.5,
        11.4, 11.3, 2.5, 29.0, 33.7,
        0.8, 32.6, 238.0
    ],
    '草原面积': [

```

```

        31908, 23754, 58, 599, 1147, 1288, 514,
        2385, 45190, 37370, 168, 405, 129,
        640, 447, 200, 1200, 1200,
        2090, 7450, 15900
    ]
}

df = pd.DataFrame(data)

# =====
# 2. 数据预处理 (Preprocessing)
# =====
# 提取数值列进行计算
features = ['森林面积', '森林覆盖率', '林木蓄积量', '草原面积']
X = df[features]

# 数据标准化 (Standardization) - 非常重要!
# 因为“森林面积”是几万，“覆盖率”是几十，不标准化会导致结果偏差。
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# =====
# 3. 聚类模型训练 (K-Means Clustering)
# =====
# 这里我们将国家分为 4 类 (K=4)，你可以尝试改成 3 或 5 查看不同效果
kmeans = KMeans(n_clusters=4, random_state=42)
df['聚类结果'] = kmeans.fit_predict(X_scaled)

# =====

```

```

# 4. 输出结果分析 (Result Analysis)
# =====
print("【聚类分析结果】")
print("-" * 50)

# 按类别分组打印
for cluster_id in sorted(df['聚类结果'].unique()):
    print(f"\n>>> 类别 {cluster_id + 1} 包含的国家:")
    countries = df[df['聚类结果'] == cluster_id]['Country'].values
    print(", ".join(countries))

    # 计算该类的平均特征, 帮助理解这一类的特点
    mean_stats = df[df['聚类结果'] == cluster_id][features].mean()
    print(f"    (特征描述: 森林面积约 {mean_stats['森林面积']:.0f}, 覆盖率约 {mean_stats['森林覆盖率']:.1f}%, 草原面积约 {mean_stats['草原面积']:.0f})")

三
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules

# 1. 加载数据
# UCI Adult 数据集通常没有表头, 我们需要手动定义
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data"
column_names = [
    "age", "workclass", "fnlwgt", "education", "education-num",
    "marital-status", "occupation", "relationship", "race", "sex",

```

```

    "capital-gain", "capital-loss", "hours-per-week", "native-country",
    "salary"
]

# 读取数据，注意处理原始数据中的空格
print("正在从 UCI 下载并读取数据...")

df = pd.read_csv(url, names=column_names, skipinitialspace=True)

# 2. 数据预处理
# 筛选收入大于 50k 的数据
df_high_income = df[df['salary'] == '>50K'].copy()

print(f"收入 >50K 的样本数量: {len(df_high_income)}")

# 筛选题目指定的 7 个变量
target_cols = [
    'workclass', 'education', 'marital-status', 'occupation',
    'relationship', 'race', 'sex'
]
df_subset = df_high_income[target_cols]

# 3. 数据转换 (One-Hot Encoding)
# Apriori 算法需要 0/1 矩阵或布尔值矩阵
# pd.get_dummies 会把 'sex' 变成 'sex_Male', 'sex_Female' 等列
df_ohe = pd.get_dummies(df_subset)

# 为了节省内存和提高可读性，将所有值转换为布尔类型 (True/False)
df_ohe = df_ohe.astype(bool)

```

```

# 4. 挖掘关联规则
print("正在挖掘频繁项集...")

# min_support: 最小支持度。设为 0.1 意味着该组合至少要在 10% 的数据中出现
# 如果结果太少，可以降低这个值；如果太慢，可以提高这个值
frequent_itemsets = apriori(df_ohe, min_support=0.15, use_colnames=True)

print("正在生成关联规则...")

# metric="lift": 使用提升度作为评估标准（也可以用 confidence）
rules = association_rules(frequent_itemsets, metric="lift",
min_threshold=1.0)

# 5. 提取前 10 条规则
# 我们按 'lift'（提升度）降序排列，提升度越高表示关联性越强（不仅仅是巧合）
top_10_rules = rules.sort_values(by='lift', ascending=False).head(10)

# 6. 格式化输出
print("\n===== 关联性排名前 10 的规则 （针对 >50K 群体）
=====")

# 重新组织列以便阅读
output_cols = ['antecedents', 'consequents', 'support', 'confidence',
'lift']
print(top_10_rules[output_cols].to_string(index=False))

# 简单的解释辅助
print("\n===== 结果解读示例 =====")

if not top_10_rules.empty:
    first_rule = top_10_rules.iloc[0]

```

```

ant = list(first_rule['antecedents'])[0]
con = list(first_rule['consequents'])[0]
print(f"第一条规则意味着：在收入>50K 的人群中，如果是 '{ant}'，")
print(f"那么大概率也是 '{con}'。")
print(f"Lift (提升度) 为 {first_rule['lift']:.2f}，表示这种关联比随机猜测强 {first_rule['lift']:.2f} 倍。")

```

2. 运行结果（截图需同时截入右下角电脑时间）：

一

```

print("-" * 50)
print(f"{'样本ID':<8} | {'KNN 预测类别':<15} | {'决策树 预测类别':<15}")
print("-" * 50)

for i in range(len(X_test)):
    print(f"{'样本ID':<8} | {'y_pred_knn':<15} | {'y_pred_dt':<15}")
    print("-" * 50)

df_result = pd.DataFrame({
    '待测样本': [1, 2, 3, 4],
    'KNN结果': y_pred_knn,
    '决策树结果': y_pred_dt
})

```

样本ID	KNN 预测类别	决策树 预测类别
样本 1	1	1
样本 2	1	1
样本 3	2	1
样本 4	2	2

二

```

# 计算该类的平均特征，帮助理解这一类的特点
mean_stats = df[df['聚类结果'] == cluster_id][features].mean()
print(f" (特征描述: 森林面积约 {mean_stats['森林面积']:.0f}, 覆盖率 {mean_stats['森林覆盖率']:.1f}%, 草原面积约 {mean_stats['草原面积']:.0f})")

```

```

>>> 类别 1 包含的国家:
加拿大, 巴西
(特征描述: 森林面积约 45056, 覆盖率约 50.1%, 草原面积约 9142)

>>> 类别 2 包含的国家:
日本, 德国, 英国, 法国, 意大利, 捷克, 波兰, 匈牙利, 南斯拉夫, 罗马尼亚, 保加利亚, 印度, 印度尼西亚, 尼日利亚, 墨西哥
(特征描述: 森林面积约 1636, 覆盖率约 31.7%, 草原面积约 1109)

>>> 类别 3 包含的国家:
中国, 美国, 澳大利亚
(特征描述: 森林面积约 17041, 覆盖率约 18.9%, 草原面积约 33617)

>>> 类别 4 包含的国家:
俄罗斯
(特征描述: 森林面积约 92000, 覆盖率约 41.1%, 草原面积约 37370)

```

三

```
first rule = top 10 rules.iloc[0]
ant = list(first rule['antecedents'])[0]
con = list(first rule['consequents'])[0]
print(f"第一条规则意味着：在收入>50K的人群中，如果是 '{ant}'")
print(f"那么大概率也是 '{con}'。")
print(f"Lift (提升度) 为 {first rule['lift']:.2f}，表示这种关联比随机要强 {first rule['lift']:.2f} 倍。")

[4] ✓ 4Bs Python

正在从 uci 下载并读取数据...
收入 >50K 的样本数量：7641
正在挖掘频繁项集...
正在生成关联规则...

===== 关联性排名前 10 的规则 (针对 >50K 群体) =====
antecedents                                consequents  support  confidence  lift
(sex_Male, race_White, marital-status_Married-civ-spouse) (occupation_Exec-managerial, relationship_Husband) 0.178804 0.257531 1.342621
(sex_Male, race_White, marital-status_Married-civ-spouse) (sex_Male, race_White, marital-status_Married-civ-spouse) 0.178804 0.932181 1.342621
(sex_Male, occupation_Exec-managerial, marital-status_Married-civ-spouse) (race_White, relationship_Husband) 0.178804 0.927652 1.340341
(sex_Male, occupation_Exec-managerial, marital-status_Married-civ-spouse) (sex_Male, occupation_Exec-managerial, marital-status_Married-civ-spouse) 0.178804 0.258290 1.340341
(sex_Male, education_HS-grad, marital-status_Married-civ-spouse) (race_White, relationship_Husband) 0.157760 0.925898 1.337503
(sex_Male, education_HS-grad, marital-status_Married-civ-spouse) (sex_Male, education_HS-grad, marital-status_Married-civ-spouse) 0.157760 0.227892 1.337503
(sex_Male, education_HS-grad, relationship_Husband) (education_HS-grad, relationship_Husband) 0.157760 0.227223 1.335572
(sex_Male, education_HS-grad, relationship_Husband) (sex_Male, education_HS-grad, relationship_Husband) 0.157760 0.927286 1.335572
(sex_Male, race_White, marital-status_Married-civ-spouse) (sex_Male, race_White, marital-status_Married-civ-spouse) 0.157760 0.927286 1.335572
(sex_Male, race_White, marital-status_Married-civ-spouse) (workclass_Private, relationship_Husband) 0.440888 0.921130 1.326704
(sex_Male, race_White, marital-status_Married-civ-spouse) (sex_Male, race_White, marital-status_Married-civ-spouse) 0.440888 0.635011 1.326704

===== 结果解读示例 =====
第一条规则意味着：在收入>50K的人群中，如果是 'sex_Male'，
那么大概率也是 'occupation_Exec-managerial'。
Lift (提升度) 为 1.34，表示这种关联比随机要强 1.34 倍。
```

三、实验的分析与思考（若有）

1. 强关联规则背后的社会学意义：家庭结构的稳定性

在挖掘出的前 10 条强关联规则中，我们可以明显观察到 marital-status（婚姻状况）与 relationship（家庭角色）在 >50K 群体中占据了核心地位。特别是 {relationship_Husband} -> {marital-status_Married-civ-spouse} 及其逆向规则，通常具有极高的置信度（Confidence）和支持度（Support）。这反映出在 1994 年的美国普查数据中，高收入群体呈现出高度的“家庭稳定性”特征。已婚且承担家庭核心责任（如 Husband）的个体，往往更容易跻身得高收入阶层。这可能与年龄因素（通常年长者收入更高且更可能已婚）有关，同时也暗示了稳定的家庭结构可能为个体职业发展提供了支持，或者高收入本身增加了组建稳定家庭的概率。

2. 人力资本与职业分层的映射：教育与职位的“锁定效应”

通过分析 education（教育）与 occupation（职业）之间的关联规则，算法验证了“人力资本理论”。我们通常会发现 {education_Bachelors/Masters} -> {occupation_Exec-managerial/Prof-specialty} 这类规则具有较高的提升度（Lift）。这表明在高收入人群中，学历与职业之间存在显著的锁定效应：高薪往往集中在管理层（Exec-managerial）和专业技术领域（Prof-specialty），而这些职位几乎都以高等教育为准入门槛。这说明在收入超过 50K 的分界线上，低学历通过体力劳动或其他途径进入高收入群体的概率被大幅压缩，教育程度成为了阶层流动最关键的变量。

3. 对算法有效性与“常识性规则”的反思

在实验结果中，我们不可避免地会发现一些“平凡规则”或“常识性规则”，例如

{relationship_Husband} $\rightarrow$ {sex_Male}。虽然这条规则的提升度 (Lift) 和置信度都很高，但它仅仅描述了定义上的逻辑关系（丈夫必然是男性），而非数据中挖掘出的隐性知识。这引发了我对关联规则挖掘“后处理”环节的思考：单纯依赖算法输出的 Lift 排序是不够的。在实际应用中，必须引入领域知识 (Domain Knowledge) 对规则进行过滤，剔除这种定义上的冗余信息，才能让分析聚焦于那些非显而易见、具有决策指导意义的“有趣模式” (Interesting Patterns)。